



中国海洋大学
OCEAN UNIVERSITY OF CHINA

第三周实验报告

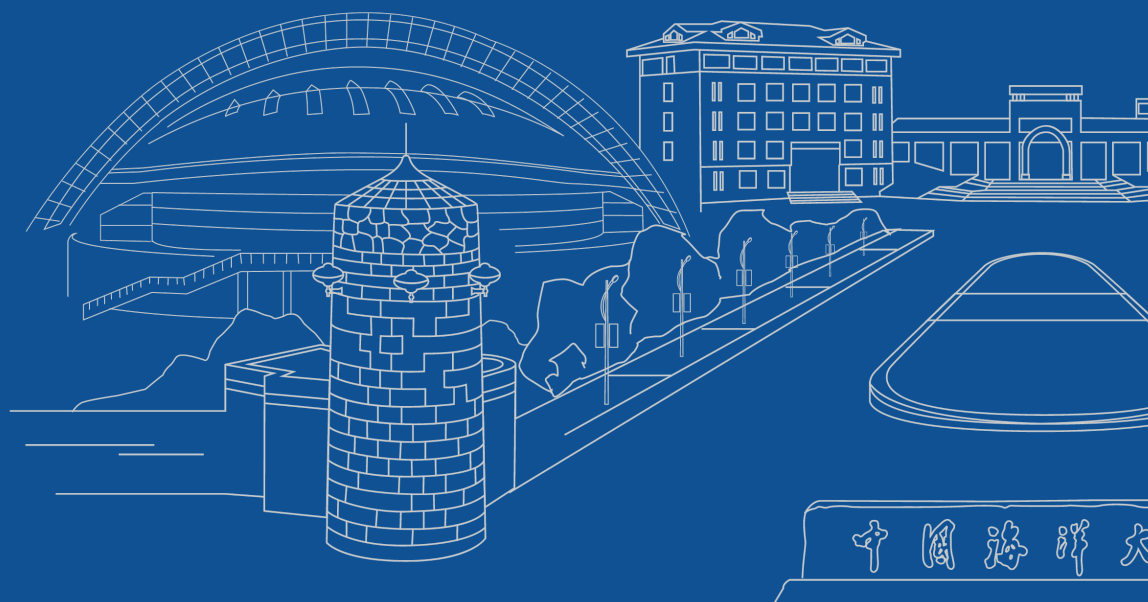
系统开发工具基础

刘晨希

25020007076

信息科学与工程学部
计算机科学与技术学院
计算机科学与技术专业
系统开发工具基础

2026 年 9 月 14 日



中国海洋大学



中国海洋大学
OCEAN UNIVERSITY OF CHINA

第三周实验报告

系统开发工具基础

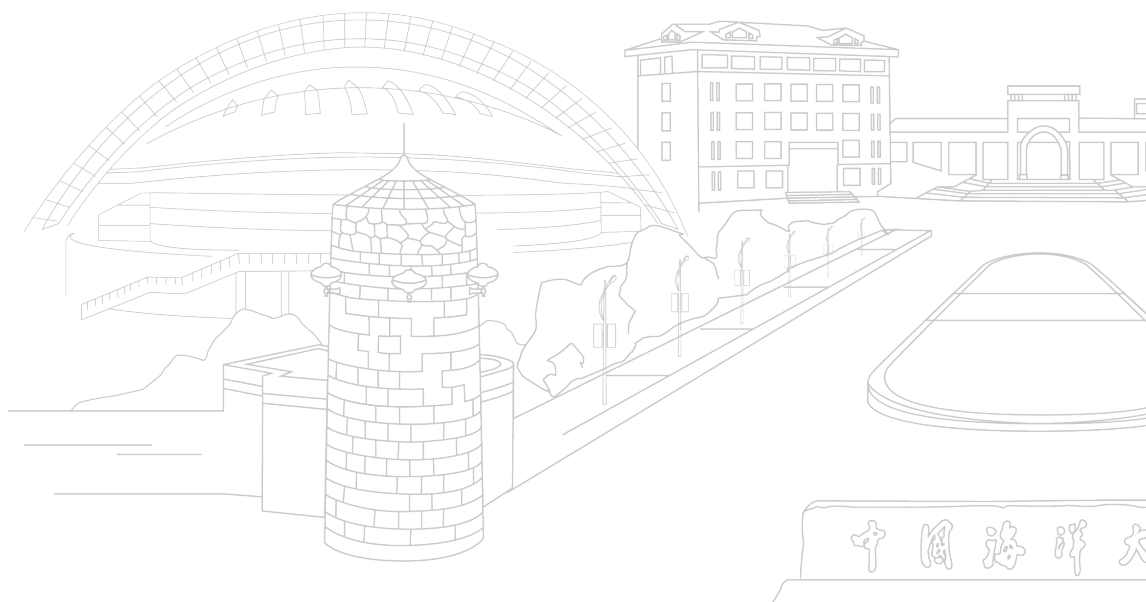
刘晨希

25020007076

指导老师: 周小伟

信息科学与工程学部
计算机科学与技术学院
计算机科学与技术专业
系统开发工具基础

2026 年 9 月 14 日



中国海洋大学

目录

目录	1
1 课堂练习及结果	3
1.1 第 9 题从源码构建并在干净环境安装 <code>Wheel</code>	3
1.1.1 实验内容	3
1.1.2 实验结果	5
1.2 第 10 题让编程智能体进入可验证的修复循环	5
1.2.1 实验内容	5
1.2.2 实验结果	8
1.3 第 11 题把”无法处理”的协作材料改成可执行信息	8
1.3.1 实验内容	8
1.3.2 实验结果	10
1.4 第 12 题修复一个可复现的线性回归训练循环	10
1.4.1 实验内容	10
1.4.2 实验结果	11
1.5 Github 仓库地址	11
2 综合实例练习：代码发布、智能体编程与 PyTorch	12
2.1 代码打包与发布（Packaging and Shipping Code）	12
题 1：用 <code>venv</code> 创建并激活隔离的虚拟环境	12
题 2：用 <code>pip</code> 安装依赖并观察传递依赖	12
题 3：用 <code>pyproject.toml</code> 与 <code>build</code> 构建 <code>wheel</code> 包	13
题 4：用 <code>pip freeze</code> 锁定依赖版本以复现环境	15
2.2 智能体编程（Agentic Coding）	16
题 5：为编程代理编写 <code>AGENTS.md</code> 说明文件	16
题 6：把一次性脚本改写成带 <code>argparse</code> 的正规命令行程序	17
题 7：用 <code>mypy</code> 进行静态类型检查	18
2.3 PyTorch 入门实例	18
题 8：张量的创建、属性与基本运算	18
题 9： <code>requires_grad</code> 与 <code>backward</code> 自动微分	19
题 10：用 <code>torch.nn</code> 定义并训练一个神经网络	19
2.4 本章小结	20

课堂练习及结果

1.1 第 9 题从源码构建并在干净环境安装 Wheel

1.1.1 实验内容

在课程仓库的 w3 目录下完成第三周的四个练习。如图 1.1 所示，在 q09 中创建 src/greetlab/__init__.py 与 src/greetlab/cli.py；cli.py 用 argparse 声明必填选项--name，在 main() 中解析参数并打印问候语。

```
liuchenxi@liuchenxideMacBook-Air q09 % mkdir -p src/greetlab
liuchenxi@liuchenxideMacBook-Air q09 % touch src/greetlab/__init__.py
liuchenxi@liuchenxideMacBook-Air q09 % cd src/greetlab
liuchenxi@liuchenxideMacBook-Air greetlab % vim cli.py
liuchenxi@liuchenxideMacBook-Air greetlab % cat cli.py
import argparse

def main():
    p = argparse.ArgumentParser()
    p.add_argument("--name", required=True)
    a = p.parse_args()
    print(f"Hello, {a.name}!")
```

图 1.1: 创建 src/greetlab 包与 cli.py

如图 1.2 所示，编写 pyproject.toml：[project] 段的 name 为 greetlab-25020007076（把题目模板中的“学号”替换为自己的学号），version 为 0.1.0；[project.scripts] 段把命令行入口 sdt-greet 指向 greetlab.cli 的 main 函数；[build-system] 段声明 setuptools 构建后端。

```
liuchenxi@liuchenxideMacBook-Air q09 % vim pyproject.toml
liuchenxi@liuchenxideMacBook-Air q09 % cat pyproject.toml
[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"
[project]
name = "greetlab-25020007076"
version = "0.1.0"
[project.scripts]
sdt-greet = "greetlab.cli:main"
```

图 1.2: 编写 pyproject.toml 声明包元数据与命令入口

由于本机未预装构建工具，如图 1.3 所示，先创建虚拟环境 `build_env` 并在其中执行 `python -m pip install build`，成功安装 `build 1.6.1` 及其依赖 `packaging`、`pyproject_hooks`；随后用 `python -m build --version` 确认工具可用。

```
((build_env) liuchenxi@liuchenxideMacBook-Air q09 % python -m pip install build
Collecting build
  Downloading build-1.6.1-py3-none-any.whl.metadata (5.6 kB)
Collecting packaging>=24.0 (from build)
  Using cached packaging-26.3-py3-none-any.whl.metadata (3.5 kB)
Collecting pyproject_hooks (from build)
  Using cached pyproject_hooks-1.2.0-py3-none-any.whl.metadata (1.3 kB)
Downloading build-1.6.1-py3-none-any.whl (31 kB)
Using cached packaging-26.3-py3-none-any.whl (129 kB)
Using cached pyproject_hooks-1.2.0-py3-none-any.whl (10 kB)
Installing collected packages: pyproject_hooks, packaging, build
Successfully installed build-1.6.1 packaging-26.3 pyproject_hooks-1.2.0
((build_env) liuchenxi@liuchenxideMacBook-Air q09 % python -m build --version
build 1.6.1 (/Users/liuchenxi/Documents/GitHub/lcx_course_work/w3/q09/build_env/lib/python3.14/site-packages/build)
```

图 1.3: 在 `build_env` 中安装 `build` 构建工具

如图 1.4 所示，在已安装构建工具的 `build_env` 中运行 `python -m build`，构建过程依次完成 `sdist` 与 `wheel`，输出 `Successfully built greetlab_25020007076-0.1.0.tar.gz` and `greetlab_25020007076-0.1.0-py3-none-any.whl`；用 `ls dist` 确认 `dist` 目录下同时生成了 `.tar.gz` 与 `.whl` 两个分发产物。

```
adding 'greetlab_25020007076-0.1.0.dist-info/RECORD'
removing build/bdist.macosx-26.0-arm64/wheel
Successfully built greetlab_25020007076-0.1.0.tar.gz and greetlab_25020007076-0.1.0-py3-none-any.whl
((build_env) liuchenxi@liuchenxideMacBook-Air q09 % ls
build_env      dist            pyproject.toml  src
((build_env) liuchenxi@liuchenxideMacBook-Air q09 % ls dist
greetlab_25020007076-0.1.0-py3-none-any.whl  greetlab_25020007076-0.1.0.tar.gz
```

图 1.4: 运行 `python -m build` 生成 `wheel` 与 `sdist` 产物

如图 1.5 所示，退出 `q09` 目录后回到 `w3`，用 `python3 -m venv q09_test_env` 新建一个干净虚拟环境，用于验证 `wheel` 能否脱离源码独立安装。

```
((build_env) liuchenxi@liuchenxideMacBook-Air q09 % cd ..
((build_env) liuchenxi@liuchenxideMacBook-Air w3 % python3 -m venv q09_test_env
```

图 1.5: 新建干净虚拟环境 `q09_test_env`

如图 1.6 所示，激活 `q09_test_env` 后执行 `pip install` 安装 `q09/dist/greetlab_25020007076-0.1.0-py3-none-any.whl`，只从本地生成的 `wheel` 安装，输出 `Successfully installed greetlab-25020007076-0.1.0`，全程没有从镜像源下载该包。

```
(q09_test_env) liuchenxi@liuchenxideMacBook-Air w3 % python -m pip install q09/dist/greetlab_25020007076-0.1.0-py3-non
l
Processing ./q09/dist/greetlab_25020007076-0.1.0-py3-none-any.whl
Installing collected packages: greetlab-25020007076
Successfully installed greetlab-25020007076-0.1.0
```

图 1.6: 在干净环境中仅从 `wheel` 安装

如图 1.7 所示，用 `pip list` 查看该环境内全部包，除 `greetlab-25020007076 0.1.0` 外只有 `pip 26.2.1` 本身，说明这是一个未受源码与构建工具污染的干净环境。

```
((q09_test_env) liuchenxi@liuchenxideMacBook-Air w3 % python -m pip list
Package            Version
-----
greetlab-25020007076 0.1.0
pip                 26.2.1
```

图 1.7: `pip list` 确认干净环境内仅装有 `greetlab` 与 `pip`

如图 1.8 所示, 切换到 q09 之外运行 `sdt-greet --name 25020007076`, 输出 `Hello, 25020007076!`, 说明 `wheel` 安装正确、`[project.scripts]` 声明的命令行入口可用。

```
[(q09_test_env) liuchenxi@liuchenxideMacBook-Air w3 % sdt-greet --name 25020007076
Hello, 25020007076!
```

图 1.8: 在 q09 之外运行 `sdt-greet` 验证入口命令

如图 1.9 所示, 用 `git add .` 与 `git commit -m "完成了 q09"` 提交, 提交 7b3fe5b 共变更 10 个文件、新增 30 行, 包含 `cli.py`、`pyproject.toml` 及构建产物。

```
[(q09_test_env) liuchenxi@liuchenxideMacBook-Air w3 % git add .
[(q09_test_env) liuchenxi@liuchenxideMacBook-Air w3 % git commit -m "完成了 q09"
[main 7b3fe5b] 完成了 q09
10 files changed, 30 insertions(+)
create mode 100644 w3/q09/dist/greetlab_25020007076-0.1.0-py3-none-any.whl
create mode 100644 w3/q09/dist/greetlab_25020007076-0.1.0.tar.gz
create mode 100644 w3/q09/pyproject.toml
create mode 100644 w3/q09/src/greetlab/__init__.py
create mode 100644 w3/q09/src/greetlab/cli.py
create mode 100644 w3/q09/src/greetlab_25020007076.egg-info/PKG-INFO
create mode 100644 w3/q09/src/greetlab_25020007076.egg-info/SOURCES.txt
create mode 100644 w3/q09/src/greetlab_25020007076.egg-info/dependency_links.txt
create mode 100644 w3/q09/src/greetlab_25020007076.egg-info/entry_points.txt
create mode 100644 w3/q09/src/greetlab_25020007076.egg-info/top_level.txt
```

图 1.9: 第九题的 `git` 提交记录

1.1.2 实验结果

第九题完成了「源码 → 构建 `sdist` 与 `wheel` → 干净虚拟环境仅从 `wheel` 安装 → 在包目录之外运行命令行入口」的完整链路: `python -m build` 成功产出 `greetlab_25020007076-0.1.0.tar.gz` 与 `greetlab_25020007076-0.1.0-py3-none-any.whl`, 新建的 `q09_test_env` 中仅有 `greetlab` 与 `pip` 两个包, `sdt-greet --name 25020007076` 正确输出 `Hello, 25020007076!`, 验证了 Python 打包与分发的整条链路。

1.2 第 10 题让编程智能体进入可验证的修复循环

1.2.1 实验内容

复用第 9 题的 `greetlab` 包, 如图 1.10 所示, 用 `cp -R q09 q10` 把 q09 复制为 q10 并查看其中的文件列表, 随后在 q10 中补一个「失败测试」: 当 `--name` 只含空白字符时, `main` 应以 `SystemExit(2)` 结束。

```
liuchenxi@liuchenxideMacBook-Air w3 % cp -R q09 q10
liuchenxi@liuchenxideMacBook-Air w3 % cd q10
liuchenxi@liuchenxideMacBook-Air q10 % find . -type f
./q09/dist/greetlab_25020007076-0.1.0.tar.gz
./q09/dist/greetlab_25020007076-0.1.0-py3-none-any.whl
./q09/build_env/bin/Activate.ps1
./q09/build_env/bin/pip3.14
./q09/build_env/bin/pip3
./q09/build_env/bin/activate.fish
```

图 1.10: 复制 q09 为 q10

如图 1.11 所示, 本机基础 Python 环境中没有 pytest, `python3 -m pytest` 直接报 No module named pytest; 因此新建虚拟环境 `test_env`, 在其中安装 `pytest 9.1.1` 及其依赖 `iniconfig`、`pluggy`、`packaging`、`pygments`。

```
liuchenxi@liuchenxideMacBook-Air q10 % python3 -m pytest --version
/opt/homebrew/opt/python@3.14/bin/python3.14: No module named pytest
liuchenxi@liuchenxideMacBook-Air q10 % python3 -m venv test_env
liuchenxi@liuchenxideMacBook-Air q10 % source test_env/bin/activate
(test_env) liuchenxi@liuchenxideMacBook-Air q10 % python -m pip install pytest
Collecting pytest
  Downloading pytest-9.1.1-py3-none-any.whl.metadata (7.6 kB)
Collecting iniconfig>=1.0.1 (from pytest)
  Downloading iniconfig-2.3.0-py3-none-any.whl.metadata (2.5 kB)
Collecting packaging>=22 (from pytest)
  Using cached packaging-26.3-py3-none-any.whl.metadata (3.5 kB)
Collecting pluggy<2,>=1.5 (from pytest)
  Downloading pluggy-1.6.0-py3-none-any.whl.metadata (4.8 kB)
Collecting pygments>=2.7.2 (from pytest)
  Downloading pygments-2.21.0-py3-none-any.whl.metadata (2.5 kB)
Downloading pytest-9.1.1-py3-none-any.whl (386 kB)
Downloading pluggy-1.6.0-py3-none-any.whl (20 kB)
Downloading iniconfig-2.3.0-py3-none-any.whl (7.5 kB)
Using cached packaging-26.3-py3-none-any.whl (129 kB)
Downloading pygments-2.21.0-py3-none-any.whl (1.3 MB)
1.3/1.3 MB 2.9 MB/s 0:00:00
Installing collected packages: pygments, pluggy, packaging, iniconfig, pytest
Successfully installed iniconfig-2.3.0 packaging-26.3 pluggy-1.6.0 pygments-2.21.0 pytest-9.1.1
```

图 1.11: 创建 `test_env` 并安装 `pytest`

如图 1.12 所示, 进入 `tests` 目录并用编辑器创建测试文件 `test_cli.py`。

```
(test_env) liuchenxi@liuchenxideMacBook-Air q10 % cd tests
(test_env) liuchenxi@liuchenxideMacBook-Air tests % vim test_cli.py
```

图 1.12: 创建测试文件 `tests/test_cli.py`

测试内容为: 用 `monkeypatch` 把 `sys.argv` 替换为 `["sdt-greet", "--name", " "]`, 断言 `main()` 抛出 `SystemExit` 且退出码为 2。如图 1.13 所示, 先运行 `python3 -m pytest` 确认测试失败: 报错 `Failed: DID NOT RAISE SystemExit`, 并捕获到标准输出为 `Hello, !`, 即空姓名仍被当作正常输入输出问候语并以 0 退出, 与期望不符, 符合“先运行测试确认失败”的要求。

```
(test_env) liuchenxi@liuchenxideMacBook-Air q10 % python3 -m pytest
platform darwin -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0
rootdir: /Users/liuchenxi/Documents/GitHub/lcx_course_work/q10
configfile: pyproject.toml
collected 1 item

tests/test_cli.py F

===== test session starts =====
platform darwin -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0
rootdir: /Users/liuchenxi/Documents/GitHub/lcx_course_work/q10
configfile: pyproject.toml
collected 1 item

tests/test_cli.py F

===== FAILURES =====
test_blank_name_exits_with_2

monkeypatch = <pytest.monkeypatch.MonkeyPatch object at 0x108795810>

    def test_blank_name_exits_with_2(monkeypatch):
        monkeypatch.setattr("sys.argv", ["sdt-greet", "--name", " "])
        with pytest.raises(SystemExit) as exc:
            #
            #
            #
        E       Failed: DID NOT RAISE SystemExit

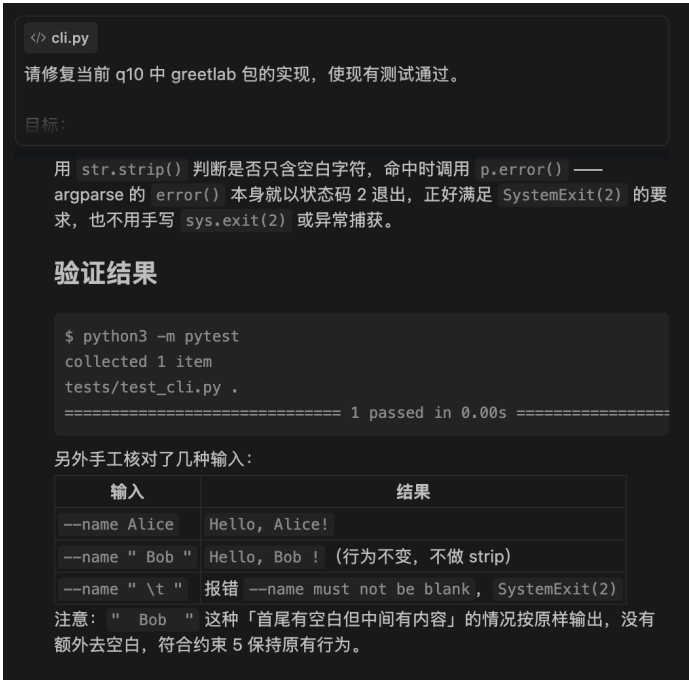
tests/test_cli.py:18: Failed

===== Captured stdout call =====
Hello, !

===== short test summary info =====
FAILED tests/test_cli.py::test_blank_name_exits_with_2 - Failed: DID NOT RAISE SystemExit
1 failed in 0.92s
```

图 1.13: 先运行测试确认失败

接下来向编程智能体说明目标 (`--name` 只含空白字符时以 `SystemExit(2)` 结束)、约束 (只修改实现、不修改测试) 和测试命令, 要求它修改实现并运行测试。如图 1.14 所示, 智能体用 `str.strip()` 判断姓名是否只含空白字符, 命中时调用 `p.error()`——`argparse` 的 `error()` 本身即以状态码 2 退出, 正好满足 `SystemExit(2)` 的要求, 无需手写 `sys.exit(2)`; 修改后测试通过, 并另外手工核对了 `--name Alice`、`--name " Bob "`、`--name "\t "` 三种输入, 确认首尾有空白但中间有内容的输入保持原有行为、不做额外 `strip`。



```

((test_env) liuchenxi@liuchenxideMacBook-Air q10 % git add .
((test_env) liuchenxi@liuchenxideMacBook-Air q10 % git commit -m "完成了q10"
[main 558417a] 完成了q10
13 files changed, 51 insertions(+)
create mode 100644 w3/q10/ai_log.md
create mode 100644 w3/q10/pyproject.toml
create mode 100644 w3/q10/src/greetlab/__init__.py
create mode 100644 w3/q10/src/greetlab/__pycache__/__init__.cpython-314.pyc
create mode 100644 w3/q10/src/greetlab/__pycache__/cli.cpython-314.pyc
create mode 100644 w3/q10/src/greetlab/cli.py
create mode 100644 w3/q10/src/greetlab_25020007076.egg-info/PKG-INFO
create mode 100644 w3/q10/src/greetlab_25020007076.egg-info/SOURCES.txt
create mode 100644 w3/q10/src/greetlab_25020007076.egg-info/dependency_links.txt
create mode 100644 w3/q10/src/greetlab_25020007076.egg-info/entry_points.txt
create mode 100644 w3/q10/src/greetlab_25020007076.egg-info/top_level.txt
create mode 100644 w3/q10/tests/__pycache__/test_cli.cpython-314-pytest-9.1.1.pyc
create mode 100644 w3/q10/tests/test_cli.py

```

图 1.17: 记录 `ai_log.md` 并提交第十题

1.2.2 实验结果

第十题完成了「测试先行失败、智能体修复、人工检查 diff、再次验证」的可验证修复循环：测试从 1 failed (DID NOT RAISE SystemExit) 变为 1 passed; `ai_log.md` 用 4 行记录完整闭环；diff 确认实现层仅为新增的空白姓名校验，测试文件未被改动，符合“只修改实现、不修改测试”的约束。

1.3 第 11 题把“无法处理”的协作材料改成可执行信息

1.3.1 实验内容

围绕“空姓名仍输出问候语”这一缺陷，改写三段质量较差的协作材料。如图 1.18 所示，原始材料存在明显问题：Issue 只写 Windows 上运行不了，尽快修复。，提交信息为 fix bug，评审意见为这里写得好，重写。，均无法据此定位与执行。

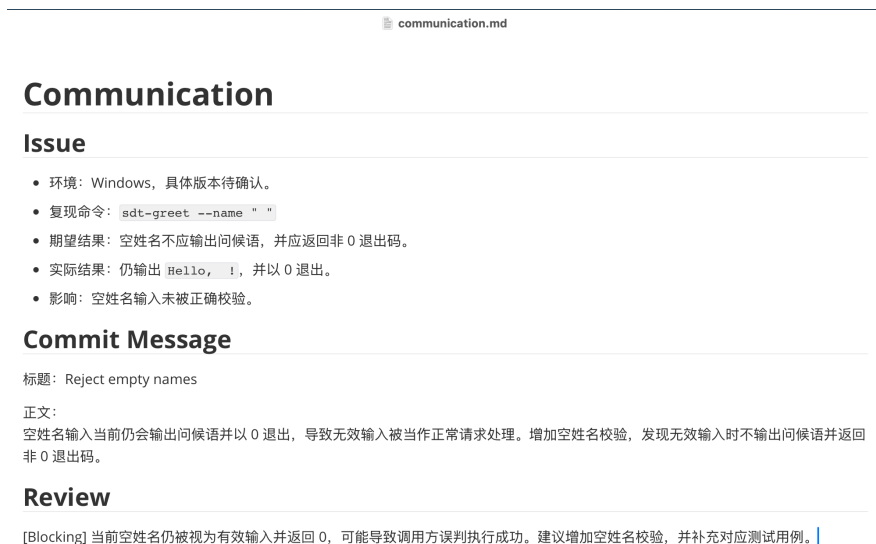
```

((test_env) liuchenxi@liuchenxideMacBook-Air q11 % vim communication.md
((test_env) liuchenxi@liuchenxideMacBook-Air q11 % cat communication.md
Issue: Windows 上运行不了，尽快修复。
提交信息: fix bug
评审意见: 这里写得不好，重写。
已知事实: 运行 sdt-greet --name " " 时仍输出 Hello, !, 并以 0 退出。

```

图 1.18: 改写前的原始协作材料

如图 1.19 所示，在 `communication.md` 中重写三部分内容：Issue 补齐环境 (Windows, 具体版本待确认)、复现命令 (`sdt-greet --name " "`)、期望结果 (空姓名不应输出问候语并应返回非 0 退出码)、实际结果 (仍输出 Hello, ! 并以 0 退出) 和影响，未知信息标注“待确认”；提交信息标题改用祈使语气 Reject empty names，正文说明问题与解决方案；评审意见指出具体行为、风险和建议动作，并标注为 Blocking。

图 1.19: 重写后的 *communication.md*

如图 1.20 所示，用 `cat communication.md` 核验最终内容，全文分为 Issue、Commit Message、Review 三节，未包含任何无法执行的空泛表述。

```
[liuchenxi@liuchenxideMacBook-Air q11 % cat communication.md]
# Communication

## Issue
- 环境：Windows，具体版本待确认。
- 复现命令：`sdt-greet --name " "`
- 期望结果：空姓名不应输出问候语，并应返回非 0 退出码。
- 实际结果：仍输出 `Hello, !`，并以 0 退出。
- 影响：空姓名输入未被正确校验。

## Commit Message
标题：Reject empty names

正文：
空姓名输入当前仍会输出问候语并以 0 退出，导致无效输入被当作正常请求处理。增加空姓名校验，发现无效输入时不输出问候语并返回非 0 退出码。

## Review
[Blocking] 当前空姓名仍被视为有效输入并返回 0，可能导致调用方误判执行成功。建议增加空姓名校验，并补充对应测试用例。
```

图 1.20: 核验 *communication.md* 最终内容

经 `wc -m` 核验，全文为 352 字，满足“全文控制在 400 字以内”的要求。如图 1.21 所示，提交 6b68cfd 变更 1 个文件、新增 17 行。

```
[liuchenxi@liuchenxideMacBook-Air q11 % git add communication.md]
[liuchenxi@liuchenxideMacBook-Air q11 % git commit -m "完成了q11"]
[main 6b68cfd] 完成了q11
1 file changed, 17 insertions(+)
create mode 100644 w3/q11/communication.md
```

图 1.21: 第十一题的 *git* 提交记录

1.3.2 实验结果

第十一题把三段“无法处理”的协作材料改写成了可执行信息, 最终 communication.md 全文 352 字 (≤ 400): Issue 补齐环境、复现命令、期望结果、实际结果与影响, 并把“Windows 具体版本”标注为待确认; 提交信息以祈使式标题 Reject empty names 开头, 正文说明问题与解决方案; 评审意见以 Blocking 标注空姓名仍被视为有效输入的风险, 并给出增加校验与补充测试用例的建议。

1.4 第 12 题修复一个可复现的线性回归训练循环

1.4.1 实验内容

如图 1.22 所示, 将给定代码保存为 q12/train.py 并补全两处 TODO: 在训练循环中依次调用 `opt.zero_grad()` 清空梯度、`loss.backward()` 反向传播、`opt.step()` 更新参数; 训练结束后调用 `model.eval()` 进入评估模式, 并在 `torch.no_grad()` 中计算最终损失、打印 weight 与 bias。训练使用 `torch.manual_seed(20260907)` 固定随机种子, 输入为 $x \in [-1, 1]$ 上的 101 个点, 目标关系为 $y = 3x - 1$, 优化器为学习率 0.1 的 SGD, 共训练 200 步。

```
(.venv) liuchenxi@liuchenxideMacBook-Air q12 % cat train.py
import torch
from torch import nn

torch.manual_seed(20260907)

x = torch.linspace(-1, 1, 101).reshape(-1, 1)
y = 3 * x - 1

model = nn.Linear(1, 1)

loss_fn = nn.MSELoss()
opt = torch.optim.SGD(model.parameters(), lr=0.1)

for _ in range(200):
    pred = model(x)
    loss = loss_fn(pred, y)

    # 清空梯度
    opt.zero_grad()

    # 反向传播
    loss.backward()

    # 更新参数
    opt.step()

# 进入评估模式
model.eval()

# 计算并打印最终结果
with torch.no_grad():
    pred = model(x)
    final_loss = loss_fn(pred, y)

    print("Final loss:", final_loss.item())
    print("Weight:", model.weight.item())
    print("Bias:", model.bias.item())
```

图 1.22: 补全 TODO 后的 q12/train.py

如图 1.23 所示, 运行 `python train.py`。运行时 torch 提示 No module named 'numpy' 的 UserWarning, 但该警告只影响 NumPy 互转, 不影响 CPU 上的张量运算与训练, 程序正常完成并输出 Final loss: 2.3923129102709773e-12、Weight: 2.99999737739563、Bias: -0.999998807907104。最终损失远小于题目要求的 0.001, weight 收敛到约 3、bias 收敛到约 -1, 且全程未直接把 weight 和 bias 赋值为 3 和 -1。

```
[(.venv) liuchenxi@liuchenxideMacBook-Air q12 % python train.py  
/Users/liuchenxi/Documents/GitHub/lcx_course_work/w3/q12/.venv/lib/python3.14/site-packages/torch/_su  
bclasses/functional_tensor.py:368: UserWarning: Failed to initialize NumPy: No module named 'numpy' (  
Triggered internally at /Users/runner/work/pytorch/pytorch/torch/csrc/utils/tensor_numpy.cpp:84.)  
  cpu = _conversion_method_template(device=torch.device("cpu"))  
Final loss: 2.3923129102709773e-12  
Weight: 2.99999737739563  
Bias: -0.999998807907104
```

图 1.23: 运行 *train.py* 的训练结果

如图 1.24 所示, 提交 bfca7fa 变更 1 个文件、新增 37 行, 即补全后的 *train.py*。

```
[liuchenxi@liuchenxideMacBook-Air q12 % git add .  
[liuchenxi@liuchenxideMacBook-Air q12 % git commit -m "完成了q12"  
[main bfca7fa] 完成了q12  
1 file changed, 37 insertions(+)  
create mode 100644 w3/q12/train.py
```

图 1.24: 第十二题的 *git* 提交记录

1.4.2 实验结果

第十二题通过补全训练循环 (*zero_grad*、*backward*、*step*) 和评估阶段的 *no_grad* 计算, 完成了一次 PyTorch CPU 线性回归训练: 最终损失 $2.39 \times 10^{-12} < 0.001$, *weight* 收敛至 $2.999997 \approx 3$, *bias* 收敛至 $-0.9999999 \approx -1$, 训练成功。运行时的 NumPy 缺失警告不影响计算结果。

1.5 Github 仓库地址

本次课程的报告、练习、代码等, 均已上传至个人 github 仓库。
Github 仓库地址: https://github.com/lcx821/lcx_course_work

综合实例练习：代码发布、智能体编程与 PyTorch

2.1 代码打包与发布 (Packaging and Shipping Code)

题 1：用 venv 创建并激活隔离的虚拟环境

要求：课程指出，不同项目的依赖必须相互隔离，避免版本冲突。使用标准库 `venv` 为本项目建立独立虚拟环境，激活后确认解释器位置与版本。

操作命令：

```
python3.14 -m venv .venv
source .venv/bin/activate
which python
python --version
```

结果说明：如图 2.1 所示，激活后 shell 提示符前出现 `(.venv)` 前缀；`which python` 指向项目目录 `.venv/bin/python` 而非系统解释器，`python --version` 输出 `Python 3.14.7`，说明后续所有 `pip` 安装都只作用于该项目环境。

```
$ python3.14 -m venv .venv
$ source .venv/bin/activate
(.venv) $ which python
/Users/liuchenxi/Documents/系统开发工具/githubclone/lcx_course_work/ms2-practice/.venv/bin/python
(.venv) $ python --version
Python 3.14.7
(.venv) $
```

图 2.1: 创建并激活 venv 虚拟环境

题 2：用 pip 安装依赖并观察传递依赖

要求：课程讲解了依赖解析与传递依赖 (transitive dependencies)。安装第三方库 `requests`，观察 `pip` 自动安装的间接依赖，并用 `pip list` 查看环境内全部包。

操作命令：

```
pip install requests
pip list
```

```
python -c 'import requests; print(requests.__version__)'
```

结果说明：如图 2.2 所示，pip 除了安装 requests 2.34.2 本身，还自动解析并安装了它依赖的 certifi、charset-normalizer、idna、urllib3 四个传递依赖；pip list 列出隔离环境内的全部 6 个包，最后导入 requests 并打印版本号，验证安装成功。

```
(.env) $ pip install requests
Looking in indexes: https://mirrors.ustc.edu.cn/pypi/simple/
Collecting requests
  Using cached requests-2.34.2-py3-none-any.whl (73 kB)
Collecting charset_normalizer<4,>=2 (from requests)
  Using cached charset_normalizer-3.5.1-cp314-cp314-macosx_10_15_universal2.whl (341 kB)
Collecting idna<4,>=2.5 (from requests)
  Using cached idna-3.19-py3-none-any.whl (68 kB)
Collecting urllib3<3,>=1.26 (from requests)
  Using cached urllib3-2.7.0-py3-none-any.whl (131 kB)
Collecting certifi>=2023.5.7 (from requests)
  Using cached certifi-2026.7.22-py3-none-any.whl (136 kB)
Installing collected packages: urllib3, idna, charset_normalizer, certifi, requests
Successfully installed certifi-2026.7.22 charset_normalizer-3.5.1 idna-3.19 requests-2.34.2 urllib3-2.7.0
(.env) $ pip list
Package            Version
-----
certifi            2026.7.22
charset-normalizer 3.5.1
idna               3.19
pip                26.2.1
requests           2.34.2
urllib3            2.7.0
(.env) $ python -c 'import requests; print(requests.__version__)'
2.34.2
(.env) $
```

图 2.2: pip 安装 requests 及其传递依赖

题 3：用 pyproject.toml 与 build 构建 wheel 包

要求：现代 Python 包通过 pyproject.toml 声明项目元数据与构建后端，执行 python -m build 可同时构建源码分发包 sdist (.tar.gz) 和预编译分发包 wheel (.whl)。wheel 本质上是一个 zip 压缩包；在 [project.scripts] 中声明的入口会在安装后成为可执行命令。

包源码 greeting.py（函数均带类型注解，并用 argparse 接收参数）：

```
def greet(name: str) -> str:
    return f"Hello, {name}!"

def main() -> None:
    import argparse
    parser = argparse.ArgumentParser(description="Greet someone")
    parser.add_argument("name", help="要打招呼的对象")
    args = parser.parse_args()
    print(greet(args.name))
```

清单文件 pyproject.toml：

```
[project]
name = "greeting"
version = "0.1.0"
requires-python = ">=3.10"

[project.scripts]
```

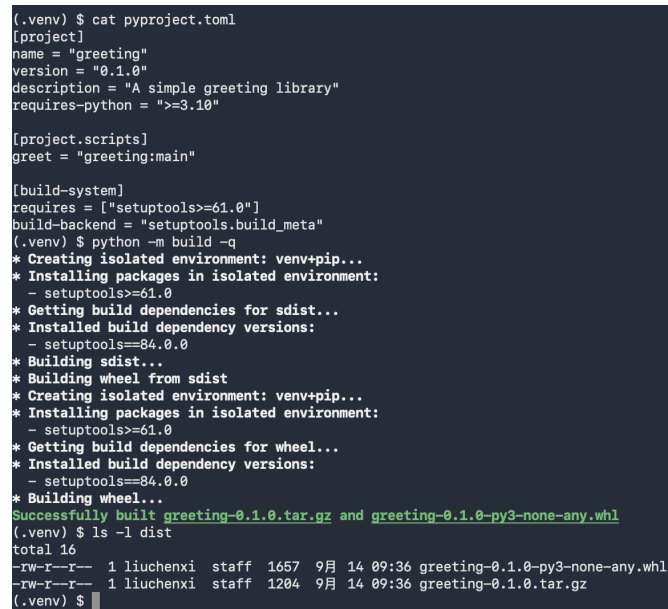
```
greet = "greeting:main"

[build-system]
requires = ["setuptools>=61.0"]
build-backend = "setuptools.build_meta"
```

构建命令（-q 为安静模式，只保留关键阶段输出）：

```
python -m build -q
ls -l dist
```

如图 2.3 所示，构建依次完成 sdist 与 wheel，最终提示 Successfully built，dist 目录下同时生成了 greeting-0.1.0.tar.gz 和 greeting-0.1.0-py3-none-any.whl 两个产物。



```
(.venv) $ cat pyproject.toml
[project]
name = "greeting"
version = "0.1.0"
description = "A simple greeting library"
requires-python = ">=3.10"

[project.scripts]
greet = "greeting:main"

[build-system]
requires = ["setuptools>=61.0"]
build-backend = "setuptools.build_meta"
(.venv) $ python -m build -q
* Creating isolated environment: venv+pip...
* Installing packages in isolated environment:
  - setuptools>=61.0
* Getting build dependencies for sdist...
* Installed build dependency versions:
  - setuptools==84.0.0
* Building sdist...
* Building wheel from sdist
* Creating isolated environment: venv+pip...
* Installing packages in isolated environment:
  - setuptools>=61.0
* Getting build dependencies for wheel...
* Installed build dependency versions:
  - setuptools==84.0.0
* Building wheel...
Successfully built greeting-0.1.0.tar.gz and greeting-0.1.0-py3-none-any.whl
(.venv) $ ls -l dist
total 16
-rw-r--r-- 1 liuchenxi staff 1657 9月 14 09:36 greeting-0.1.0-py3-none-any.whl
-rw-r--r-- 1 liuchenxi staff 1204 9月 14 09:36 greeting-0.1.0.tar.gz
(.venv) $
```

图 2.3: 构建 sdist 与 wheel 产物

查看 wheel 内容、安装并运行入口命令：

```
unzip -l dist/greeting-0.1.0-py3-none-any.whl
pip install -q ./dist/greeting-0.1.0-py3-none-any.whl
greet World
greet Alice
```

如图 2.4 所示，wheel 内除了 greeting.py，还有 METADATA、entry_points.txt、RECORD 等元数据文件，共 6 个文件；本地安装后，[project.scripts] 声明的 greet 命令可直接在终端运行，分别输出 Hello, World! 与 Hello, Alice!，完成了「源码 → 构建 → 分发包 → 安装运行」的完整发布闭环。

```
(.venv) $ unzip -l dist/greeting-0.1.0-py3-none-any.whl
Archive: dist/greeting-0.1.0-py3-none-any.whl
  Length      Date    Time    Name
-----
  389   09-14-2026  01:34   greeting.py
  111   09-14-2026  01:36   greeting-0.1.0.dist-info/METADATA
   91   09-14-2026  01:36   greeting-0.1.0.dist-info/WHEEL
   40   09-14-2026  01:36   greeting-0.1.0.dist-info/entry_points.txt
    9   09-14-2026  01:36   greeting-0.1.0.dist-info/top_level.txt
  463   09-14-2026  01:36   greeting-0.1.0.dist-info/RECORD
-----
 1103                      6 files
(.venv) $ pip install -q ./dist/greeting-0.1.0-py3-none-any.whl
(.venv) $ greet World
Hello, World!
(.venv) $ greet Alice
Hello, Alice!
(.venv) $
```

图 2.4: 查看 wheel 内容、安装并运行 `greet` 命令

题 4: 用 `pip freeze` 锁定依赖版本以复现环境

要求: 课程强调环境可复现性: 把当前环境中每个依赖的精确版本导出为锁定文件, 其他机器即可据此还原出完全一致的环境。

操作命令:

```
pip freeze > requirements.txt
wc -l requirements.txt
cat requirements.txt
```

结果说明: 如图 2.5 所示, `pip freeze` 共导出 24 个包, 每一行都以包名 == 精确版本的形式固定 (如 `torch==2.14.0`、`requests==2.34.2`), 连传递依赖也一并锁定。在新环境中执行 `pip install -r requirements.txt` 即可复现同一套依赖。

```
(.venv) $ pip uninstall -y greeting -q
(.venv) $ pip freeze > requirements.txt
(.venv) $ wc -l requirements.txt
    24 requirements.txt
(.venv) $ cat requirements.txt
ast_serialize==0.11.2
build==1.6.1
certifi==2026.7.22
charset-normalizer==3.5.1
filelock==3.32.6
fsspec==2026.7.0
idna==3.19
Jinja2==3.1.6
librt==0.15.0
MarkupSafe==3.0.3
mpmath==1.3.0
mypy==2.3.1
mypy_extensions==1.1.0
networkx==3.6.1
numpy==2.5.3
packaging==26.3
pathspec==1.1.1
pyproject_hooks==1.2.0
requests==2.34.2
setuptools==84.0.0
sympy==1.14.0
torch==2.14.0
typing_extensions==4.16.0
urllib3==2.7.0
(.venv) $
```

图 2.5: `pip freeze` 生成版本锁定文件

2.2 智能体编程 (Agentic Coding)

题 5：为编程代理编写 AGENTS.md 说明文件

要求：课程介绍，编程代理启动时会自动读取项目根目录下的 `AGENTS.md`（或 `CLAUDE.md`），把项目环境与工作约定预载进上下文，从而无需每次重复交代。用 `heredoc` 创建该文件并回显确认。

操作命令：

```
cat > AGENTS.md <<'EOF'
# AGENTS.md: 给编程代理的项目说明（启动时自动加载到上下文）
## 环境
# Python 3.14, 依赖装在 .venv, 先 source .venv/bin/activate
## 工作规则
# 1. 修改任何 .py 文件后必须运行 mypy 类型检查
# 2. 所有函数必须带类型注解
# 3. 打包验证命令: python -m build
EOF
cat AGENTS.md
```

结果说明：如图 2.6 所示，文件中写明了解释器与虚拟环境位置，以及「改完即做类型检查、函数必须带类型注解、打包验证命令」三条规则；`cat` 回显确认内容已正确写入。

```
(.venv) $ cat > AGENTS.md <<'EOF'
# AGENTS.md: 给编程代理的项目说明 (启动时自动加载到上下文)
## 环境
# Python 3.14, 依赖装在 .venv, 先 source .venv/bin/activate
## 工作规则
# 1. 修改任何 .py 文件后必须运行 mypy 类型检查
# 2. 所有函数必须带类型注解
# 3. 打包验证命令: python -m build
EOF
(.venv) $ cat AGENTS.md
# AGENTS.md: 给编程代理的项目说明 (启动时自动加载到上下文)
## 环境
# Python 3.14, 依赖装在 .venv, 先 source .venv/bin/activate
## 工作规则
# 1. 修改任何 .py 文件后必须运行 mypy 类型检查
# 2. 所有函数必须带类型注解
# 3. 打包验证命令: python -m build
(.venv) $
```

图 2.6: 创建 AGENTS.md 并回显确认

题 6: 把一次性脚本改写成带 argparse 的正规命令行程序

要求: 课程演示了把随手脚本逐步改造成「可被人和代理可靠调用」的正规 CLI: 用 argparse 声明位置参数与选项、自动生成帮助信息, 并为函数补充类型注解。编写 `extract\_links.py` 从 Markdown 文件提取链接, 支持 `--absolute` 选项只保留绝对 URL。

核心代码:

```
def extract(content: str) -> list[str]:
    return LINK_PATTERN.findall(content)

def main() -> None:
    parser = argparse.ArgumentParser(description="从 Markdown 文件提取链接")
    parser.add_argument("path", type=Path, help="Markdown 文件路径")
    parser.add_argument("--absolute", action="store_true", help="只保留绝对 URL")
    args = parser.parse_args()
    content: str = args.path.read_text(encoding="utf-8")
    links = extract(content)
    if args.absolute:
        links = [url for url in links if url.startswith("http")]
    for link in links:
        print(link)
```

运行命令:

```
python extract_links.py --help
python extract_links.py sample.md
python extract_links.py sample.md --absolute
```

结果说明: 如图 2.7 所示, `--help` 自动生成了用法、位置参数 `path` 和选项 `--absolute` 的说明; 默认运行提取出 3 条链接 (含 1 条相对链接 `./local.md`), 加上 `--absolute` 后只保留 2 条 `http` 开头的绝对链接, 过滤行为符合预期。

```
(.venv) $ python extract_links.py --help
usage: extract_links.py [-h] [--absolute] path

从 Markdown 文件提取链接

positional arguments:
  path                Markdown 文件路径

options:
  -h, --help          show this help message and exit
  --absolute          只保留绝对 URL
(.venv) $ python extract_links.py sample.md
https://pytorch.org
./local.md
https://missing-semester-cn.github.io
(.venv) $ python extract_links.py sample.md --absolute
https://pytorch.org
https://missing-semester-cn.github.io
(.venv) $
```

图 2.7: argparse 命令行程序的帮助与两种运行方式

题 7：用 mypy 进行静态类型检查

要求：课程强调「类型检查反馈环」——每次改完代码就让静态检查器代理自动发现类型问题。对上一实例中带完整类型注解的脚本运行 mypy。

操作命令：

```
mypy extract_links.py
```

结果说明：如图 2.8 所示, mypy 输出 Success: no issues found in 1 source file, 说明脚本的类型注解自治、没有类型错误, 这正是 AGENTS.md 中「修改后必须运行 mypy」规则的一次实际执行。

```
(.venv) $ mypy extract_links.py
Success: no issues found in 1 source file
(.venv) $
```

图 2.8: mypy 静态类型检查通过

2.3 PyTorch 入门实例

题 8：张量的创建、属性与基本运算

要求：对应教程「开始使用 PyTorch」, 创建张量并查看 shape、dtype、device 属性, 练习特殊张量、逐元素运算与矩阵乘法。

操作命令：

```
python tensors.py
```

结果说明：如图 2.9 所示，从列表创建的 2×2 浮点张量属性为 `shape: (2, 2)`、`dtype: torch.float32`、`device: cpu`；`torch.zeros(2,3)` 生成全零张量，`x + 10` 对每个元素加 10（广播），`x.sum()` 归约为标量 10.0，`x @ I` 与单位矩阵相乘后仍等于 `x`。

```
(.venv) $ python tensors.py
x =
tensor([[1., 2.],
        [3., 4.]])
shape: (2, 2) dtype: torch.float32 device: cpu
zeros =
tensor([[0., 0., 0.],
        [0., 0., 0.]])
x + 10 =
tensor([[11., 12.],
        [13., 14.]])
x.sum() = 10.0
x @ I =
tensor([[1., 2.],
        [3., 4.]])
(.venv) $
```

图 2.9: PyTorch 张量创建、属性与运算

题 9: requires_grad 与 backward 自动微分

要求：对应 autograd 教程：对需要求导的叶子张量设置 `requires\_grad=True`，前向计算后调用 `backward()`，梯度自动累积到 `.grad`，并与手算结果对照。取 $y = wx + b$ 、 $\text{loss} = y^2$ ，其中 $w = 2$ ， $x = 3$ ， $b = 1$ 。

操作命令：

```
python autograd_demo.py
```

结果说明：如图 2.10 所示，前向得到 $y = 7$ 、 $\text{loss} = 49$ ；反向传播后 $\partial \text{loss} / \partial x = 28$ 、 $\partial \text{loss} / \partial w = 42$ 、 $\partial \text{loss} / \partial b = 14$ 。以 x 为例手算 $\partial \text{loss} / \partial x = 2y \cdot w = 2 \times 7 \times 2 = 28$ ，与自动微分结果完全一致。

```
(.venv) $ python autograd_demo.py
y = 7.0 loss = 49.0
d(loss)/dx = 28.0 (手算 2*y*w = 28.0 )
d(loss)/dw = 42.0
d(loss)/db = 14.0
(.venv) $
```

图 2.10: autograd 自动微分并与手算对照

题 10: 用 torch.nn 定义并训练一个神经网络

要求：对应教程「使用 torch.nn 创建并训练神经网络」：用 `nn.Sequential` 堆叠线性层与 ReLU，搭配均方误差损失和 SGD 优化器，按「清零梯度 → 前向 → 反向 → 更新参数」的训练循环拟合目标函数 $y = 3x - 2$ 。

核心代码：

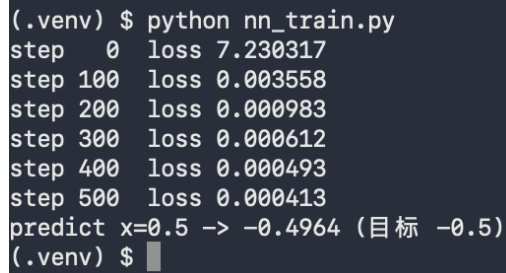
```
model = nn.Sequential(nn.Linear(1, 16), nn.ReLU(), nn.Linear(16, 1))
loss_fn = nn.MSELoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.05)

for step in range(501):
    optimizer.zero_grad()
    y_pred = model(X)
    loss = loss_fn(y_pred, y_true)
    loss.backward()
    optimizer.step()
```

操作命令：

```
python nn_train.py
```

结果说明：如图 2.11 所示，每 100 步打印一次损失，loss 从初始的 7.230317 持续下降到 0.000413；训练结束后对 $x = 0.5$ 预测得到 -0.4964 ，与目标值 $3 \times 0.5 - 2 = -0.5$ 几乎一致，说明网络成功学到了线性关系。



```
(.venv) $ python nn_train.py
step 0 loss 7.230317
step 100 loss 0.003558
step 200 loss 0.000983
step 300 loss 0.000612
step 400 loss 0.000493
step 500 loss 0.000413
predict x=0.5 -> -0.4964 (目标 -0.5)
(.venv) $
```

图 2.11: *nn.Sequential* 网络的训练损失与预测结果

2.4 本章小结

本章 10 个实例覆盖了三条主线：其一，代码发布侧实践了虚拟环境隔离、pip 依赖解析与传递依赖、基于 `pyproject.toml` 的 `wheel` 构建—安装—运行闭环，以及用 `pip freeze` 锁定可复现环境；其二，智能体编程侧实践了用 `AGENTS.md` 向代理预置项目约定、用 `argparse` 与类型注解把脚本工程化、用 `mypy` 建立自动类型检查反馈环；其三，PyTorch 侧完成了张量运算、`autograd` 自动微分（与手算互相验证）和 `torch.nn` 完整训练循环。所有操作均在终端逐条执行并保留了命令—输出截图，实验过程可按本章命令完整复现。

解题感悟

第三周的实验围绕 Python 打包发布、智能体协作和机器学习训练展开，相比前两周的 Shell 命令和调试，这周的实验明显更接近真实的工程开发流程，也更有挑战。

第 9 题让我第一次完整走了一遍“源码 - 构建 - 安装 - 运行”的打包流程。一开始对 `pyproject.toml` 里的 `[build-system]`、`[project]`、`[project.scripts]` 这些配置完全不熟悉，不知道 `sdt-greet` 这个命令是从哪里来的。后来才明白 `[project.scripts]` 就是把包里的函数注册成命令行入口，构建之后 `pip` 安装，系统就能直接调用。第 9 题要求“只从 `wheel` 安装”，我一开始想在干净环境里直接 `pip install greetlab`，结果当然装不上，后来才理解题目的用意：只有把构建产物本身装进去，才能证明打包是正确的。

第 10 题是让我感触最深的一题。题目要求让编程智能体进入“可验证的修复循环”：先写测试确认失败，再让智能体修复，最后人工检查 `diff`。我一开始是让智能体直接改代码，改完就跑，结果代码虽然能运行，但我自己都不知道改了什么、为什么改。按照题目的流程做下来，先看到 `TypeError` 的失败输出，再看到智能体补上 `argv` 参数和空白校验，最后用 `diff` 确认只有 `cli.py` 有改动，整个过程每一步都有证据，非常踏实。我也第一次意识到，测试不是“多写的作业”，而是让修改可验证的锚点；`ai_log.md` 用几行字把提示、改动和验证记录下来的习惯也值得坚持。

第 11 题字数限制 400 字看起来不难，真正写的时候才发现把问题讲清楚还要控制字数并不容易。我一开始写 `Issue` 的时候把复现步骤写得过于冗长，又漏掉了环境信息，初版写完是 403 字还超了。精简的时候学会了把“环境、复现、期望、实际”这类信息用最少的词说清楚，把未知信息诚实地标注成“待确认”而不是编一个结论。这让我意识到，协作材料最重要的是让接收方“能直接执行”，而不是堆字数。

第 12 题是第一次用 `PyTorch` 写训练循环。题目里把 `zero_grad`、`backward`、`step` 都留成了 `TODO`，补全本身不难，难的是理解为什么要 `zero_grad`：不清空梯度的话，多次 `backward` 会把梯度累加，参数更新就会出错。运行出 `final loss: 2.39e-12` 的时候很有成就感，也体会到机器学习调试“看指标、找原因、改逻辑”的基本思路。

这周的实验比前两周更“工程化”：打包、测试、协作、训练，每一步都有明确的产物和验证标准。虽然过程依然有卡壳的时候（比如环境变量没生效、`numpy` 缺失的警告），但按照“先验证、再修改、最后核对”的流程，问题都能定位到具体环节。希望在后面的课程里能把这种工作方式保持下去。

